



Logging

Tim Schmitz
16.05.2026

Inhalt

Voraussetzungen	3
Konfiguration	3
Level	4
Deklaration	4
Kodierung	4

Logging

Logging ist für ein qualifiziertes Debuggen des Quellcodes und für eine Traceability der Anwendung, d.h. die Nachvollziehbarkeit von Fehlern unerlässlich. Es gibt verschiedene Logger, darunter auch einen im JDK selbst. Als Logger wird hier der frei verfügbare Logger des Apache Subprojects log4j2 verwendet. Dieser Logger zeichnet sich durch eine weite Verbreitung, eine hohe Stabilität und eine große Flexibilität sowie eine entsprechende Konfigurierbarkeit aus.

Voraussetzungen

Um die Klassen des Loggers zu nutzen, müssen folgende zwei Archive und das Properties-Verzeichnis in den Klassenpfad eingebunden werden. Dies gilt sowohl für die IDE als auch für den Build und Deployment-Prozess mit einem Build-Tool.

- log4j-api-2.11.1.jar
- log4j-core-2.11.1.jar
- properties/log4j2.xml

Das properties-Verzeichnis muss deswegen miteingebunden werden, um auf die Konfigurationsdatei log4j2.xml zugreifen zu können.

Konfiguration

Um den oder die Logger zu konfigurieren, gibt es mehrere Möglichkeiten. Die hier vorgestellte Möglichkeit basiert auf der Konfiguration mittels einer XML-Datei log4j2.xml. Diese soll dann verwendet werden, wenn nicht die Standard-Konfiguration (Default Configuration) des Loggers verwendet werden soll, andererseits aber auch keine Configuration Factory oder kein System-Property zum Einsatz kommt.

Die Konfigurationsdatei log4j2.xml hat folgendes Aussehen:

```
<?xml version="1.0" encoding="UTF-8"?>
<Configuration status="WARN">
  <!-- ***** -->
  <!-- Definition der Appender Console und Log File -->
  <!-- ***** -->
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{HH:mm:ss:SSS} %-5level %logger{36} - %msg%n" />
    </Console>

    <File name="File"
      fileName="${sys:log.path}/output_${date:yyyy}_${date:MM}_${date:dd}_${date:HH}
        _${date:mm}_${date:ss}_${date:SSS}.log"
      bufferedIO="true">
      <PatternLayout pattern="%d{HH:mm:ss:SSS} %-5level %logger{36} - %msg%n" />
    </File>
  </Appenders>

  <!-- ***** -->
  <!-- Definition der Logger -->
  <!-- ***** -->
  <Loggers>
    <Root level="trace">
      <AppenderRef ref="File" />
      <AppenderRef ref="Console" />
    </Root>
  </Loggers>
</Configuration>
```

Es handelt sich um einen Logger, der in Form von zwei Appendern auftritt, die über spezielle XML-Tags gekennzeichnet sind.

1. Als Console, die den geloggten Output auf der Kommandozeile oder in der Entwicklungsumgebung ausgibt (IDE oder Build-Messages)
2. Als Logging File, welches in einem Datei-Format mit dem Suffix .log gespeichert wird

Das Muster der geloggten Ausgabe wird in der Konfigurationsdatei festgelegt und ist immer dasselbe:

```
23:35:58:920 TRACE de.domain.application.control.MainRun - LogPath: C:\Users\\Daten\Java\Applications\application\logs
```

Zunächst wird der Zeitstempel des Events geloggt, also wann der geloggte Code-Teil durchlaufen wurde. Danach wird der Log-Level (zum Beispiel TRACE) protokolliert. Anschließend folgt der Name der vollqualifizierten Klasse, in die geloggt wurde und zum Schluss die eigentliche Mitteilung.

Level

Es gibt verschiedene Logging-Level. Sie besitzen eine implizite Hierarchie:

Level	Gewichtung
OFF	0
FATAL	100
ERROR	200
WARN	300
INFO	400
DEBUG	500
TRACE	600
ALL	Int.MAX_VALUE

Tabelle 1: Logging-Level

Setzt man beispielsweise INFO als Level des Logging Levels an, so werden alle Loggings mit dem Level ERROR übernommen, nicht aber die mit dem Level TRACE. Um den Überblick nicht zu verlieren, sollte man nicht zu viele Logging-Level verwenden.

Deklaration

Wie bereits in der Anwendungsarchitektur skizziert, wird der Logger in der Klasse MainRun gesetzt, sobald der Computer-Name ermittelt wurde. Abhängig von diesem wird der Logging-Pfad gesetzt und die für die Konfigurationsdatei System-Property log.path. Dieses geschieht in der Methode setLogPath() der erwähnten Klasse.

```
public static String setLogPath(String computerName) {
    String logPath = "";

    // Abfrage des Rechnernamens und setzen des Logging Pfads
    if (TIM_PC.equals(computerName)) {
        logPath = LOG_PATH_TIM_PC;
    } else if (META_PC.equals(computerName)) {
        logPath = LOG_PATH_META_PC;
    } else if (_PC.equals(computerName)) {
        logPath = LOG_PATH_PC;
    } else {
        System.err.println(ERROR_HOST);
    }

    // Setzen des System Logging-Pfades
    System.setProperty("log.path", logPath);
    return logPath;
}
```

Kodierung

Um den Logger im Quellcode zu verwenden, müssen drei Handlungen vorgenommen werden:

1. Die Klassenbibliotheken des Loggers und des Logging-Managers müssen importiert werden
2. Der Logger muss mit dem Namen der geloggten Klasse instantiiert werden
3. Die eigentliche Logging-Mitteilung muss an der Stelle im Code geschehen, die protokolliert werden soll.

Dafür bettet sich das Logging harmonisch in den übrigen Java-Code ein. Es hat beispielsweise folgendes Aussehen:

```
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;
```

```
public class Control {  
    private static Logger LOG = LogManager.getLogger(Control.class.getName());  
    protected static void run() {  
        LOG.trace("Start der Anwendungssteuerung");  
    }  
}
```